



Defect Prevention, Reduction & Containment

PPT PRESENTED BY

NAME : Srinivas Rao Tammireddy

ROLL NO : 257Y1D5805

YEAR : I Year I Semester

ACADEMIC YEAR : 2025 - 2026

QA Classification: Dealing with Defects

Quality Assurance consists of activities that guarantee software matches quality goals. We classify QA operations based on how they handle defects: Prevention, Reduction, or Containment.

The Cost of Defects

- **Cost Multiplier:** The cost of fixing bugs grows exponentially as software moves from design to code, to test, and finally to production.
- **Phase of Origin:** Most defects originate during requirements and design phases, but are only detected during system testing.
- **QA Goal:** Introduce defect handling activities early in the lifecycle to minimize cost and risk.

Three Pillars of Defect QA

- 1. Defect Prevention:** Stop bugs from entering the code. Focuses on requirements clarity, design rules, and developer training.
- 2. Defect Reduction:** Find and remove bugs present in code. Uses inspection (static reviews) and execution-based testing.
- 3. Defect Containment:** Limit the damage of bugs that escape into production. Focuses on fault tolerance and safety containment.

Defect Prevention

Defect Prevention is the most cost-effective QA strategy, acting before bugs are coded:

Developer Training & Spec Clarity

- **Training & Education:** Educating developers on secure coding standards and common architectural bugs (e.g., buffer overflows).
- **Formal Specifications:** Using formal mathematical specification languages (e.g. Z notation) to define system interfaces, preventing requirements ambiguity.
- **Root Cause Analysis (RCA):** Analyzing past bugs to identify process improvements, updating design rules to prevent recurrence.

Process & Technology Tools

- **Design Rules & Checklists:** Enforcing coding conventions, strict design patterns, and type checking constraints.
- **Cleanroom Software Engineering:** A process model that uses formal design verification and mathematical proofs to ensure defect-free builds before testing begins.
- **Automated Linting:** Run code analyzers during development to catch syntax errors and poor style patterns automatically.

Defect Reduction: Inspection & Testing

Defect reduction detects and removes bugs that have already been coded:

Inspections (Static Verification)

- **Non-Execution Review:** Reviews design docs, source code, and schemas without running the application.
- **Peer Code Reviews:** Team walkthroughs and formal code inspections led by moderators.
- **High Fault Detection Rate:** Inspections typically detect 60% to 90% of code defects, finding root errors (faults) directly in the source.
- **Static Analyzers:** Automated tools scan code for logic errors, dead blocks, and compliance warnings.

Software Testing (Dynamic Validation)

- **Execution-Based:** Evaluates system behavior by executing the code with test cases and inputs.
- **Failure Observation:** Observes system failures (deviations from expectations), which must be debugged to locate the root fault.
- **Testing Levels:** Ranges from Unit Testing (components) to Integration Testing (modules) and System Testing (entire application).
- **Limits:** 'Testing shows the presence of bugs, not their absence' (Dijkstra).

Defect Containment: Production Resilience

Defect containment limits the impact of bugs that escape into production:

Fault Tolerance (Software Resilience)

- **System Continuity:** The software detects runtime errors and handles them without crashing the entire system.
- **N-Version Programming:** Executes multiple versions of the same module in parallel. A voting system compares results to select the correct output, bypassing individual code bugs.
- **Recovery Blocks:** Tries to run a primary module. If the check fails, it rolls back state and executes an alternate backup module.

Safety Assurance & Containment

- **Safety Checks:** Code monitors parameters to detect dangerous conditions (e.g. medical devices checking dose values).
- **Fail-Safe Design:** If a critical failure occurs, the system defaults to a safe state (e.g., shutting down power to prevent damage).
- **Containment Boundaries:** Isolates subsystems so a crash in a non-critical component (e.g., user profile UI) cannot crash the primary transaction module.

Applications of QA Strategies

How modern dev teams implement the three pillars of QA:

CI/CD Automated Pipelines

Linting checks run on code commit (Prevention). Automated unit tests run on compilation, and integration tests run on build packaging (Reduction).

Rigorous Pull Request Code Reviews

Code reviews and checklists verify that every line of code is inspected by colleagues before merge. Eliminates logic bugs and improves quality (Reduction).

High-Availability Server Clusters

Database replicas and load balancers detect hardware or software errors and automatically redirect user traffic, preserving availability (Containment).

Embedded Control Systems

Medical, automotive, and avionics software run concurrent redundant processors to vote on mechanical decisions, preventing accidents (Containment).



THANK YOU

Department of Computer Science and Engineering
Marri Laxman Reddy Institute of Technology and Management